

Rol QA Automatizador en el Proyecto Versia Renal

Guía para desempeñar como QA en Versia Renal. Explica qué debe aprender, qué herramientas usar, cómo crear casos de prueba, cómo ejecutarlos, cómo documentar evidencias y cómo empezar a automatizar dentro del framework del proyecto.

ÁREA Calidad de Software
ROL QA Automatizador
PROYECTO Versia Renal
VERSIÓN Guía inicial

Resumen

El rol QA Automatizador en Versia Renal combina conocimiento funcional del producto, diseño de pruebas, ejecución manual, documentación de evidencias y automatización web. La persona que asuma el rol debe entender cómo se comporta la aplicación, cómo se organizan los casos en Azure Test Plans y ALM Test, y cómo se transforma un caso manual en un escenario automatizado usando Java, Selenium, Cucumber y Gherkin.

Esta guía está escrita como documento práctico de entrada. No sustituye la formación funcional ni la revisión con el equipo, pero permite seguir un camino ordenado para empezar a trabajar, comprender las herramientas y ejecutar tareas de QA con criterio.

Enfoque: el documento se centra en el flujo corporativo del proyecto: Azure DevOps, Azure Test Plans, ALM Test, evidencias y framework de automatización.

Índice

- 01** **Objetivo y alcance**
Qué cubre esta guía y cómo debe usarse.
- 02** **Rol QA Automatizador**
Responsabilidades y expectativas principales.
- 03** **Herramientas corporativas**
Azure DevOps, Azure Test Plans, ALM Test y ALM SaaS.
- 04** **Tecnologías necesarias**
Java, Selenium, Cucumber, Gherkin, Spring, Maven y navegador.
- 05** **Conocimiento funcional**
Módulos, reglas de negocio y validaciones.
- 06** **Gestión de pruebas manuales**
Test plans, suites, test cases, steps y evidencias.
- 07** **Azure Test Plans**
Cómo crear planes, suites, casos y ejecutar pruebas.
- 08** **ALM Test / ALM SaaS**
Uso, evidencias y consideraciones tras migración.
- 09** **Framework de automatización**
Estructura del proyecto y patrón Page Object.
- 10** **Creación de escenarios**
De test case manual a feature Cucumber.
- 11** **Ejecución y validación**
Cómo probar localmente y revisar resultados.
- 12** **Depuración de fallos**
HTML, XPath, iframes, Kendo UI, alerts, logs y screenshots.

- 13** Buenas prácticas
Criterios para mantener pruebas estables y legibles.
-

- 14** Checklist de entrega
Validaciones antes de cerrar un trabajo.
-

- 15** Ruta de aprendizaje
Camino sugerido para empezar desde cero.
-

1. Objetivo y alcance

El objetivo de esta guía es explicar cómo trabaja un QA Automatizador dentro del proyecto Versia Renal. La guía permite conocer qué herramientas se usan, cómo se documentan las pruebas, cómo se preparan los datos, cómo se ejecutan los casos y cómo se automatizan los flujos repetitivos.

El documento está orientado a una persona que ya tenga una base de QA o desarrollo, pero que no conozca aún el proyecto. Por eso se explican tanto las responsabilidades del rol como el flujo práctico que debe seguirse en el día a día.

2. Rol QA Automatizador

Responsabilidades principales

- Analizar requisitos, incidencias o cambios funcionales.
- Diseñar casos de prueba positivos, negativos y de cancelación.
- Crear y mantener test cases en Azure Test Plans o ALM Test.
- Ejecutar pruebas manuales cuando la funcionalidad es nueva o inestable.
- Automatizar los casos repetibles y de regresión.
- Adjuntar evidencias claras para cada ejecución relevante.
- Reportar defectos con pasos reproducibles y resultado esperado.
- Mantener el framework y corregir pruebas cuando cambia la interfaz.

Criterio esperado

- Entender el flujo funcional antes de automatizar.
- No crear automatización sobre pasos que no han sido validados manualmente.
- Usar nombres claros en features, escenarios, steps y métodos.
- Evitar duplicar código si ya existe un método reutilizable.
- Revisar las capturas, el HTML y el comportamiento real de la pantalla.
- Confirmar que el caso deja el sistema en un estado controlado.

3. Herramientas corporativas

Herramienta	Uso dentro del rol	Qué debe conocer el QA
Azure DevOps	Plataforma corporativa para trabajo del proyecto.	Ubicar el proyecto, acceder a Test Plans, consultar trabajo relacionado y moverse por el entorno.
Azure Repos	Repositorio corporativo del framework y código relacionado.	Conocer ramas, cambios, revisiones y ubicación del framework.
Azure Test Plans	Creación, organización y ejecución de pruebas.	Crear Test Plan, suites, test cases, steps, ejecuciones y evidencias.
ALM Test / ALM SaaS	Gestión histórica o actual de casos, steps y evidencias según flujo definido.	Consultar test cases, revisar evidencias, crear pasos y validar información migrada.
Microsoft Teams / Outlook	Comunicación operativa con equipo, responsables y sistemas.	Reportar bloqueos, pedir datos, validar incidencias y coordinar pruebas.

4. Tecnologías necesarias

Automatización

- **Java:** lenguaje base del framework.
- **Selenium WebDriver:** interacción con navegador y elementos web.
- **Cucumber:** ejecución de escenarios escritos en lenguaje Gherkin.
- **Gherkin:** formato Given/When/Then para describir escenarios.
- **Spring / Spring Context:** inyección de pages, steps y configuración.
- **Maven:** gestión de dependencias y ejecución del proyecto.

Análisis de interfaz

- **HTML / DOM:** lectura de estructura de pantalla.
- **XPath y CSS selectors:** localización de elementos.
- **Chrome DevTools:** inspección, búsqueda y depuración.
- **Kendo UI:** grids, dropdowns, multiselects y controles dinámicos.
- **Iframes:** cambio de contexto para acceder a formularios internos.
- **Alerts y modales:** manejo de confirmaciones del navegador y de la aplicación.

5. Conocimiento funcional del producto

Antes de crear pruebas, el QA debe entender qué hace el módulo, qué datos modifica, qué permisos intervienen y cuál es el resultado esperado. En Versia Renal existen módulos de pacientes, configuración, prescripciones, tratamientos, accesos, laboratorios, medicación, notificaciones y otros flujos clínicos u operativos.

El conocimiento funcional se obtiene revisando documentación, capturas, HTML de pantallas, casos existentes, ejecuciones previas y validación con el equipo. Un caso automatizado sin entendimiento funcional puede pasar técnicamente pero validar algo incorrecto.

6. Gestión de pruebas manuales

La prueba manual es el primer paso cuando el flujo es nuevo, cambió la interfaz o no se conoce el comportamiento exacto. El QA debe crear pasos claros, ejecutar, adjuntar evidencias y confirmar el resultado antes de decidir si el caso se automatiza.

Paso	Acción	Resultado esperado
1	Identificar el módulo y el flujo que se debe probar.	El alcance de la prueba queda claro.
2	Crear test case con título descriptivo.	El caso puede ser entendido por otro miembro del equipo.
3	Escribir steps con acción y expected result.	La ejecución es reproducible.
4	Ejecutar el caso y adjuntar evidencias.	Existe prueba visual o documental del resultado.
5	Reportar defectos si el resultado difiere del esperado.	El equipo puede reproducir y corregir la incidencia.

7. Azure Test Plans

Azure Test Plans se usa para organizar las pruebas por plan, suites, test cases y steps. La estructura debe ser clara y consistente con el resto del proyecto.

Estructura: Test Plan > Suite > Test Case > Steps > Evidencias.

Flujo recomendado

1. Entrar en Azure DevOps y abrir el proyecto Versia Renal - Comercial .
2. Abrir Test Plans .
3. Crear o abrir el Test Plan correspondiente al módulo.
4. Crear suites para agrupar funcionalidad: View, Create, Modify, Delete, Filters, Permissions, etc.
5. Crear los test cases dentro de su suite.
6. Escribir steps en inglés si ese es el estándar del proyecto.
7. Ejecutar manualmente y adjuntar capturas.
8. Marcar o asociar automatización cuando exista feature Cucumber implementado.

8. ALM Test / ALM SaaS

ALM Test o ALM SaaS puede usarse para consultar, gestionar o conservar casos de prueba, steps y evidencias según el flujo de la empresa. Tras una migración, es importante verificar que los steps y evidencias históricas siguen disponibles; si falta información, se debe comunicar al responsable correspondiente con detalle.

Cuando se trabaje con ALM, el QA debe asegurar que el caso tenga nombre claro, steps reproducibles, resultado esperado y evidencias adjuntas. Si se detecta pérdida de datos o diferencia entre ALM antiguo y ALM SaaS, debe registrarse como incidencia operativa.

9. Framework de automatización

El framework de Versia Renal está orientado a pruebas web automatizadas. La estructura habitual separa escenarios Gherkin, steps Java y métodos de página mediante Page Object Model.

Elemento	Función	Ejemplo de responsabilidad
Feature	Archivo Gherkin con escenarios.	Describe el caso funcional con Given, When, Then y And.
Steps	Clase Java que conecta Gherkin con código.	Llama métodos de Page para ejecutar acciones.
Page	Clase Page Object con locators y acciones.	Contiene XPath, WebElements y métodos reutilizables.
BasePage	Métodos comunes del framework.	Clicks, waits, dropdowns Kendo, alerts, frames y utilidades.
Resources	Datos o referencias auxiliares.	HTML de pantallas, propiedades, logs o información de apoyo.

10. Creación de escenarios automatizados

Un escenario automatizado debe salir de un caso manual entendido y probado. Primero se valida el flujo en la aplicación, después se crea el feature y finalmente se implementan los steps y métodos de Page necesarios.

Proceso de trabajo

1. Leer el test case manual y confirmar datos de prueba.
2. Crear el archivo feature en el módulo correspondiente.
3. Nombrar el escenario con el mismo objetivo del test case.
4. Reutilizar steps existentes cuando sea posible.
5. Crear nuevos steps solo si el comportamiento no existe.
6. Definir locators en la parte superior de la Page siguiendo el patrón del proyecto.
7. Implementar métodos pequeños y reutilizables.
8. Probar primero en una suite de prueba controlada.
9. Si funciona, incluir el caso en regression o en la suite que corresponda.

11. Ejecución y validación

La ejecución puede ser individual o por suite. Al terminar, el QA debe revisar resultado, logs, screenshots y reportes. Un caso no debe considerarse listo solo porque ejecutó una vez; debe ser estable, claro y no depender de tiempos o datos frágiles.

Antes de ejecutar

- Confirmar usuario y permisos.
- Confirmar datos de prueba.
- Revisar si el caso modifica o borra información.
- Asegurar que el navegador y entorno están disponibles.

Después de ejecutar

- Revisar reportes y screenshots.
- Confirmar que el resultado funcional es correcto.
- Documentar fallos reproducibles.
- Limpiar o restaurar datos si el caso lo requiere.

12. Depuración de fallos

Los fallos más comunes en automatización web no siempre son errores funcionales. Pueden deberse a tiempos de carga, iframes, elementos dinámicos, dropdowns Kendo, alerts del navegador o XPath's frágiles.

Problema	Cómo analizarlo	Solución habitual
Elemento no encontrado	Inspeccionar HTML y verificar iframes.	Cambiar al frame correcto y usar XPath estable.
Dropdown Kendo no selecciona	Verificar si la lista se renderiza fuera del contenedor.	Usar helper global de Kendo o localizar opción visible por texto.
Click cierra formulario	Revisar doble carga, timing o evento no estable.	Esperar estado correcto, repetir click si el flujo lo requiere y documentar incidencia.
Alert nativa bloquea Selenium	Confirmar si es modal interno o alert del navegador.	Usar accept/dismiss alert y cerrar frames correctamente.
XPath frágil	Revisar si depende de índices dinámicos.	Basarlo en títulos, labels, columnas o texto estable.

13. Buenas prácticas

- Usar nombres de steps directos, sin parámetros si el step solo se utiliza en un caso concreto.
- Separar locators, steps y lógica de Page según el patrón existente.
- No duplicar métodos si ya existe una utilidad común.
- No automatizar con datos que puedan desaparecer sin control.
- Usar waits explícitos en lugar de depender solo de pausas.
- Cerrar iframes con el patrón correcto del proyecto.
- Documentar defectos cuando se detecte comportamiento anormal de la aplicación.
- No subir archivos generados, reportes o HTML locales si no forman parte de la entrega.

14. Checklist de entrega

Validación	Estado esperado
Test case manual creado	Título, steps y expected result claros.
Evidencias adjuntas	Capturas suficientes para justificar el resultado.
Feature creado	Escenario con nombre y tags correctos.
Steps implementados	Reutilizables y coherentes con el estilo del proyecto.
Page actualizada	Locators arriba, métodos claros y sin duplicación innecesaria.
Ejecución local	El caso pasa en entorno controlado.
Reportes revisados	No hay fallos ocultos en logs o screenshots.
Entrega limpia	No se incluyen archivos temporales o generados.

15. Ruta de aprendizaje inicial

Para una persona que empieza desde cero, se recomienda avanzar por fases. Primero debe comprender el producto y las herramientas, luego ejecutar pruebas manuales, después leer el framework y finalmente automatizar casos pequeños antes de tocar flujos críticos.

Fase	Actividad	Resultado esperado
1	Revisar esta guía, accesos y herramientas.	Comprende el flujo general del puesto.
2	Ejecutar casos manuales existentes con evidencias.	Aprende la aplicación y la documentación de pruebas.
3	Leer features, steps y pages ya implementados.	Comprende el patrón del framework.
4	Automatizar un caso simple de consulta o cancelación.	Puede crear cambios pequeños con bajo riesgo.
5	Automatizar un flujo CRUD completo.	Puede trabajar sobre regresión de forma autónoma.